RideX Capstone Project Report (Expanded)

1. Executive Summary

RideX is a comprehensive, real-time ride-sharing application designed to connect riders with drivers efficiently. The system is distributed across three distinct user interfaces—a Rider App, a Driver App, and an Admin Dashboard—all powered by a robust, real-time Node.js backend.

2. System Architecture & Component Analysis

The project follows a modern decoupled micro-frontend architecture:

Frontend Components (React & Vite)

The frontends share a consistent layout and routing strategy, utilizing React Router DOM for navigation:

- **Layouts**:
  - `AuthLayout`: Wraps public-facing pages (Login, Register).
  - `MainLayout`: Wraps authenticated pages, providing consistent navigation bars and sidebars.
- **Role-Based Access Control (RBAC)**:
  - `ProtectedRoute`: A crucial component that wraps sensitive routes (e.g., `<ProtectedRoute allowedRoles={['driver']}>`). It intercepts unauthorized access and redirects users based on their JWT token claims.
- **Key Dashboards**:
  - `RiderDashboard`: Integrates `Mapbox GL` for interactive maps, allowing riders to drop pins for pickup/drop-off locations and view live driver tracking.
  - `DriverDashboard`: Interfaces with Socket.io to listen for incoming ride requests and provides tools for accepting, rejecting, or updating the status of a trip.
  - `AdminDashboard`: Aggregates platform data and monitors users.

Backend API (Node.js & Express)

The backend acts as a central nervous system handling business logic, database interactions, authentication, and real-time socket connections.

3. API Routes & Real-time Events

The backend exposes distinct RESTful endpoints and Socket.io event listeners.

REST API Routes

Authentication (`/api/auth`)

- `POST /register/user` - Registers a new rider/admin.
- `POST /register/driver` - Registers a new driver.
- `POST /login` - Authenticates a user and returns a JWT.
- `PUT /profile` - (Protected) Updates the authenticated user's profile.

Rides (`/api/rides`)

- `POST /book` - (Protected) Submits a new ride request to the database.
- `PUT /:id/accept` - (Protected) Driver accepts a specific ride.
- `PUT /:id/status` - (Protected) Updates the lifecycle state of a ride (e.g., 'started', 'completed').
- `GET /history` - (Protected) Retrieves past rides for the authenticated user.

Real-Time Socket.io Events

Real-time state is coordinated in-memory on the backend and broadcasted via channels:

- `join-rider` / `join-driver`: Assigns the connected socket to a specific room for targeted messaging. Drivers also join a global `'drivers'` fleet room.
- `request-ride`: Emitted by a rider. The backend broadcasts a `ride-dispatch` event to the `'drivers'` room and triggers an email/simulated SMS notification.
- `accept-ride`: Emitted by a driver. The backend assigns the ride (first-come, first-serve basis), emits `ride-accepted` to the specific rider, and `ride-withdrawn` to all other drivers.
- `reject-ride`: Triggers a `ride-rejected` event back to the rider if no driver is available.
- `complete-ride`: Marks the ride as completed in memory and notifies the rider.

4. Technology Stack

Backend

- **Node.js & Express.js**: RESTful API development.
- **MongoDB & Mongoose**: NoSQL database for flexible data storage.
- **Socket.io**: Real-time bidirectional event-based communication.
- **JWT & bcrypt**: Secure authentication and password hashing.
- **Nodemailer**: Email service integration.

Frontend (Rider, Driver, Admin)

- **React 19 & Vite**: Fast, modern frontend framework.
- **Tailwind CSS v4**: Utility-first CSS framework for responsive design.
- **Redux Toolkit**: Centralized state management.
- **Framer Motion**: Fluid animations and transitions.
- **Mapbox GL**: Interactive maps and geospatial routing.
- **Socket.io-client**: Real-time connection to the backend.

5. Core Models & Database Schema

The MongoDB database relies on the following primary collections:

- **User**: Stores rider and admin profiles (`name`, `email`, `password`, `phone`, `role`).
- **Driver**: Stores driver details, vehicle information, and current location/availability status.
- **Ride**: Tracks the complete lifecycle of a ride trip (`riderId`, `driverId`, `pickupLocation`, `dropLocation`, `status`, `fare`, `distance`).
- **Payment**: Manages transaction records.

6. Local Development Setup

Start the four distinct servers concurrently:

1. **Backend Server**: Runs on `http://localhost:5000`
2. **Rider App**: Runs on `http://localhost:5173`
3. **Driver App**: Runs on `http://localhost:5174`
4. **Admin Dashboard**: Runs on `http://localhost:5175`